

Universitat Politècnica de Catalunya
ETSETB / FIB

Motor de Recomendación Cinematográfica de Altas Prestaciones Basado en Redes Neuronales y Metadatos Heterogéneos

Propuesta de Proyecto: Caracterización de Datos Masivos, Revisión del Estado del Arte y Justificación Tecnológica

Computació d'Altes Prestacions i Intel · ligència Artificial (CPIA)

Autores:
Arnau Montull
Uriel Martínez



7 de octubre de 2026

1. Definición del Problema a Resolver

1.1. La Perspectiva del Usuario: La Fatiga de Elección

En la actualidad, uno de los mayores inconvenientes al abrir una plataforma de contenidos audiovisuales es la llamada fatiga de decisión: los usuarios a menudo invierten más tiempo navegando por menús y carruseles interminables que disfrutando de una película. La mayoría de las recomendaciones que aparecen en pantalla suelen percibirse como poco atractivas, irrelevantes o repetitivas, mostrando una y otra vez los mismos éxitos comerciales.

El usuario busca descubrir obras que encajen verdaderamente con su gusto personal, incluyendo cine de autor o títulos periféricos de calidad, pero el volumen inmanejable de los catálogos modernos convierte la búsqueda manual en una experiencia lenta y poco satisfactoria.

1.2. Dificultades Técnicas al Diseñar la Solución

Hacer una recomendación acertada y personalizada no es trivial debido a las limitaciones estructurales que presentan los datos a gran escala:

1. **Casi nadie ha visto casi nada (Esparcidad masiva):** En un catálogo con más de 80 000 títulos, una persona promedio evalúa solo unas pocas decenas o cientos de obras. Esto significa que más del 99.8 % de la matriz de interacciones carece de datos. Los algoritmos clásicos de filtrado colaborativo tienen dificultades para inferir relaciones cuando coinciden tan pocas valoraciones entre usuarios.
2. **Siempre se recomienda lo mismo (Sesgo de popularidad):** El análisis de los datos revela que el 10 % de las películas comerciales concentra más del 95 % de todas las calificaciones. Como consecuencia directa, los modelos estándar tienden a recomendar de forma redundante títulos muy votados (*blockbusters*) para maximizar su probabilidad de acierto global, invisibilizando miles de películas valiosas de la cola larga (*long-tail*).
3. **El reto de los usuarios reales y el arranque en frío (*Cold-Start*):** Cuando se añade una película recién estrenada o se incorpora un usuario nuevo, el sistema carece de historial acumulado. Un caso representativo en la comunidad de ciencia de datos es el proyecto de Sam Learner en Kaggle (*Letterboxd Movie Ratings Data*), donde se recopilaron las valoraciones de los 4 000 usuarios más activos de Letterboxd. Cuando usuarios reales intentan conectar su perfil para recibir recomendaciones basadas en su diario personal (*diary*), los modelos basados únicamente en IDs de usuario colapsan si no son capaces de procesar secuencias arbitrarias y asociarlas con los metadatos y sinopsis del catálogo.
4. **Tiempos de respuesta lentos (Latencia y cómputo en CPU):** Comparar el perfil de un usuario contra decenas de miles de películas evaluando redes neuronales completas en tiempo real genera cuellos de botella inasumibles para un servicio interactivo. La solución exige desacoplar el cálculo pesado, precomputar los vectores de las películas y emplear paralelismo multihilo y vectorización en CPU para responder en pocos milisegundos.

1.3. Objetivo del Sistema

En lugar de plantear una regresión para predecir la nota exacta de cada película, el sistema se enfoca como un problema de **Aprendizaje de Ordenamiento (*Learning to Rank*) y Recuperación Top-K**.

Formalmente, el sistema proyecta tanto al usuario u como a cada película i a un espacio métrico latente compartido $\mathbb{R}^d$ ($d \ll M$). A partir de estas representaciones se calcula una puntuación de afinidad $\hat{s}(u, i)$ para ordenar el catálogo, asegurando que las obras relevantes para el usuario ($\mathcal{I}_u^+$) se sitúen por delante del resto del inventario no observado o irrelevante:

$$\hat{s}(u, i) > \hat{s}(u, j) \quad \forall\, i \in \mathcal{I}_u^+,\ j \in \mathcal{I} \setminus \mathcal{I}_u^+ \tag{1}$$

El propósito final es devolver una selección reducida, diversa y personalizada de películas en tiempo real, ahorrando tiempo de búsqueda al espectador.

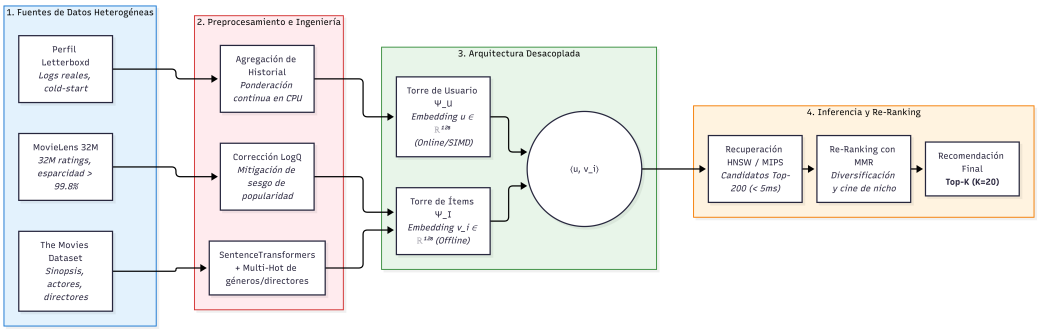


Figura 1: Esquema conceptual del problema: integración de datos masivos y heterogéneos, modelado desacoplado y pipeline de inferencia eficiente en dos fases.

2. Identificación y Caracterización de las Fuentes de Datos

Para abordar simultáneamente el modelado colaborativo, el arranque en frío y el sesgo de popularidad, el proyecto recurre a la combinación y cruce relacional de tres conjuntos de datos heterogéneos (Tabla 1):

Tabla 1: Especificaciones cuantitativas de los conjuntos de datos analizados.

Fuente de Datos	Usuarios (N)	Ítems (M)	Volumen / Registros	Dispersión
MovieLens 32M	200 948	84 432	32 000 204 ratings	99.811 %
The Movies Dataset (TMDB)	—	45 466	Metadatos y sinopsis	—
Letterboxd Export Format	Perfil real	Catálogo	Secuencia temporal	Específico

2.1. MovieLens 32M: Núcleo Cuantitativo Colaborativo

Publicado por GroupLens Research en su versión oficial más exhaustiva [1], recoge el comportamiento continuado de usuarios hasta octubre de 2023. Proporciona:

- `ratings.csv` ($\approx$ 700 MB): Registros estructurados en cuádruplas (userId, movieId, rating, timestamp). Las calificaciones abarcan una escala ordinal de 10 niveles discretos (de 0.5 a 5.0 con incrementos de 0.5 estrellas).
- `links.csv`: Tabla relacional fundamental que asocia el identificador interno `movieId` con las claves de `imdbId` y `tmdbId`, actuando como puente de enlace hacia la información semántica.

2.2. Análisis Empírico de la Distribución de Datos

El procesamiento directo de los 32 millones de registros revela dos propiedades estadísticas críticas que condicionan el diseño del sistema (Figura 2):

1. **Concentración extrema en la cola pesada (*Long-Tail*):** Del catálogo evaluado (84 432 películas con al menos un voto), el 10 % superior de títulos más populares (8 443 filmes) concentra el **95.70 % de todas las interacciones del sistema**. El 90 % restante del inventario se reparte únicamente el 4.30 % de las calificaciones registradas. Cualquier modelo optimizado sin regularización de popularidad tenderá a ignorar este 90 % del inventario.
2. **Asimetría de calificaciones (*Positive Skew*):** Las notas otorgadas presentan una clara asimetría hacia valores positivos: las notas ≥ 3.5 representan la gran mayoría del volumen

total. Esto confirma que el usuario tiende a ver y calificar títulos que espera disfrutar, haciendo aconsejable tratar las notas altas como retroalimentación implícita de afinidad y ponderar el historial.

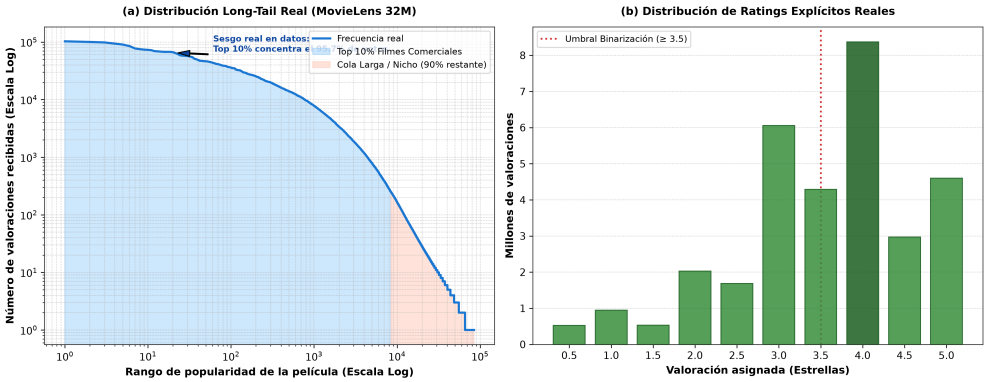


Figura 2: Distribución empírica obtenida sobre MovieLens 32M: (a) curva de potencia con el 95.70 % de interacciones concentradas en el Top 10 %, y (b) histograma de calificaciones con sesgo positivo.

2.3. The Movies Dataset (TMDB) y Metadatos Semánticos

Para dotar de contexto intrínseco al sistema y solventar el arranque en frío, se integran los metadatos de The Movie Database:

- **Atributos descriptivos y sinopsis (`movies_metadata.csv`):** Texto narrativo (*overview*), año de estreno, presupuestos y géneros cinematográficos.
- **Cuerpo técnico y reparto (`credits.csv`):** Directores, guionistas, equipo de fotografía y actores principales representados en estructuras tabulares deserializadas.
- **Palabras clave argumentales (`keywords.csv`):** Descriptores temáticos para enriquecer el perfil semántico.

2.4. Letterboxd Export Format: Caso de Uso en Entorno Real

Para validar el sistema en un entorno real de explotación, se adopta el esquema de exportación de Letterboxd (`watched.csv`, `ratings.csv`, `diary.csv`). Esto permite recrear el escenario en el que un usuario nuevo introduce su diario cinematográfico personal cronológico, permitiendo medir la latencia y la calidad de la recomendación ante perfiles no incluidos en el entrenamiento inicial.

Tabla 2: Esquema de integración y normalización relacional de atributos.

Dataset Origen	**Campo Crudo**	**Transformación / Ingeniería de Atributos**	**Uso en el Modelo**
MovieLens	`movieId`, `rating`	Binarización implícita ponderada por r_{ui}	Torre de Usuario
TMDB	`overview`	Embedding semántico denso ($d = 384$) con MiniLM	Torre de Ítems
TMDB	`genres`, `director`	Codificación dispersa Multi-Hot	Torre de Ítems
MovieLens	`links.csv`	Mapeo relacional (`tmdbId` ↔ `movieId`)	Fusión de datos
Letterboxd	`diary.csv`	Secuencia cronológica y agregación online	Inferencia en tiempo real

3. Estado del Arte y Soluciones Habituales

Para entender cómo abordar el catálogo de MovieLens y TMDB sin incurrir en cuellos de botella de memoria o latencia, revisamos las soluciones más representativas utilizadas en la literatura y en la industria:

3.1. ¿Qué se hace habitualmente? Enfoques y Casos Reales

1. **Factorización Matricial (SVD / ALS) — Caso Netflix Prize:** Popularizado por Koren et al. [2] durante la célebre competición del *Netflix Prize*. Funciona bien descomponiendo la matriz de valoraciones numéricas en factores latentes, pero es un modelo estrictamente colaborativo: ignora por completo el texto de las sinopsis de TMDB y no puede recomendar nada a un usuario nuevo sin historial previo (*cold-start*).
2. **Autoencoders Lineales (EASE^R) — Caso Harald Steck (Netflix):** Propuesto por Harald Steck [7] en Netflix y muy utilizado como solución de referencia en competiciones de Kaggle por su precisión inmediata. Su gran inconveniente con nuestro catálogo es de memoria: exige calcular e invertir una matriz de tamaño catálogo × catálogo ($M \times M$). Con más de 84 000 películas, almacenar dicha matriz requiere más de 28 GB de memoria RAM, haciendo inviable su ejecución en un ordenador convencional.
3. **Redes Neuronales Acopladas (NeuMF / NCF):** Introducido por He et al. [4], combina vectores latentes con capas densas no lineales. Aunque es un referente académico, en producción presenta un cuello de botella crítico: para recomendar películas debe pasar al usuario y al ítem juntos por toda la red. Evaluar esa red neuronal 84 000 veces por cada usuario en tiempo real destruye cualquier posibilidad de respuesta interactiva rápida.
4. **Modelos sobre Grafos (LightGCN):** Propuesto por He et al. [9], modela usuarios y películas como nodos interconectados de un grafo bipartito. Si bien captura muy bien la estructura comunitaria, mantener millones de aristas dispersas en memoria dificulta una paralelización limpia y vectorizada en CPU bajo recursos limitados.
5. **Arquitectura de Dos Torres (*Two-Tower*) — Caso YouTube / Google:** Diseñada por Covington et al. [3] y formalizada por Yi et al. [8] para el motor de recomendaciones a gran escala de YouTube. Separa el sistema en dos ramas independientes: una procesa al usuario y otra los metadatos de las películas. Esto permite precomputar los vectores de todo el catálogo una sola vez y resolver la búsqueda en milisegundos mediante índices vectoriales rápidos en memoria RAM (como FAISS de Meta).
6. **Diversificación con MMR — Caso Motores de Búsqueda y Noticias:** Formulado originalmente por Carbonell y Goldstein [10], es el estándar para evitar que la lista final sea monótona o redundante (por ejemplo, recomendar cinco secuelas de la misma saga), reequilibrando afinidad con variedad temática y cine de nicho.

Tabla 3: Comparativa práctica de paradigmas frente a las restricciones del proyecto.

Enfoque	Referencia / Caso Real	Cold-Start	Metadatos (TMDB)	Viabilidad en CPU
SVD / ALS	Netflix Prize [2]	Nulo	No compatible (solo notas)	Alta (lineal)
EASE^R	Harald Steck (Netflix) [7]	Nulo	No compatible	Inviable (> 28 GB RAM)
NeuMF	He et al. [4]	Bajo	Posible pero costoso	Muy lenta en tiempo real
LightGCN	He et al. [9]	Bajo	Requiere grafos complejos	Media (acceso disperso)
Two-Tower	YouTube / Google [8]	**Excelente**	**Nativo (texto y géneros)**	**Óptima (SIMD / FAISS)**

4. Justificación Tecnológica del Enfoque Propuesto

Para responder a las limitaciones observadas en los datos (dispersión del 99.8 %, arranque en frío y concentración del 95.70 % en superventas) garantizando respuesta en milisegundos en CPU, se

propone un flujo en **dos etapas desacopladas** (Figura 3):

4.1. Etapa 1: Recuperación Rápida con Red de Dos Torres (*Two-Tower*)

El modelo separa la representación del usuario y de las películas en dos redes independientes que proyectan a un espacio común de dimensión $d = 128$:

$$\hat{s}(u, i) = \langle \mathbf{u}, \mathbf{v}_i \rangle = \Psi_U(\mathbf{x}_u)^\top \Psi_I(\mathbf{x}_i) \tag{2}$$

Esta separación resuelve tres problemas clave del catálogo:

1. **Arranque en frío resuelto por metadatos:** La Torre de Ítem (Ψ_I) genera el vector $\mathbf{v}_i$ combinando sinopsis textuales (vía *SentenceTransformers*) con directores y géneros de TMDB. Una película recién estrenada sin votos obtiene una representación válida de inmediato.
2. **Inferencia en milisegundos sobre 84 000 películas:** Los vectores $\mathbf{v}_i$ de todo el catálogo se precomputan offline y se indexan en RAM (apenas 43 MB). En tiempo real solo se evalúa la Torre de Usuario (Ψ_U) para obtener $\mathbf{u}$ y extraer los $K_c = 200$ títulos más afines en menos de 5 ms mediante búsqueda vectorial aproximada (*HNSW*).
3. **Mitigación del sesgo de popularidad (LogQ):** Durante el entrenamiento contrastivo, se resta la probabilidad marginal de aparición del filme ($s^* = \mathbf{u}^\top \mathbf{v}_i - \tau \log P(i)$) según Yi et al. [8]. Esto evita que los títulos comerciales acaparen artificialmente las recomendaciones.

4.2. Etapa 2: Diversificación y Descubrimiento con MMR

La primera etapa prioriza relevancia pura, lo que suele devolver películas muy similares entre sí (por ejemplo, varias entregas de la misma saga). Para introducir variedad y rescatar cine de calidad de la cola larga, se aplica Relevancia Marginal Máxima [10] sobre los 200 candidatos:

$$i^* = \arg\max_{i \in \mathcal{C} \setminus \mathcal{S}} \left[\lambda \cdot \text{Score}(u, i) - (1 - \lambda) \max_{j \in \mathcal{S}} \text{Sim}(\mathbf{v}_i, \mathbf{v}_j) \right] \tag{3}$$

Fijando $\lambda \approx 0.7$, el algoritmo equilibra afinidad personal y novedad temática, produciendo la lista final de $K = 20$ recomendaciones.

5. Plan de Computación Paralela y Optimización HPC (CPIA)

El diseño traslada el cómputo crítico a operaciones vectoriales continuas en CPU, ideales para explotación multihilo y vectorización:

1. **Paralelización multihilo con OpenMP en C/C++:** El cálculo concurrente de perfiles de usuario y la evaluación masiva de similitudes se distribuirá entre núcleos con balanceo dinámico [11]:

```
#pragma omp parallel for schedule(dynamic, 64)
for (int u = 0; u < num_usuarios; ++u) {
    calcular_perfil_usuario(&usuarios[u], items_db, dim_embedding);
    evaluar_similitudes_simd(&usuarios[u], candidatos, top_k);
}
```

2. **Vectorización SIMD (AVX2 / AVX-512):** Dado que los embeddings tienen tamaño fijo ($d = 128$), los productos escalares $\langle \mathbf{u}, \mathbf{v}_i \rangle$ se optimizarán en memoria alineada usando instrucciones intrínsecas FMA (8 a 16 operaciones float por ciclo por núcleo).
3. **Evaluación de escalabilidad:** En las siguientes entregas se medirá el *Speedup* experimental $S_p = T_1/T_p$ y la eficiencia para $p \in \{1, 2, 4, 8, 16\}$ hilos frente a la cota teórica de la Ley de Amdahl.

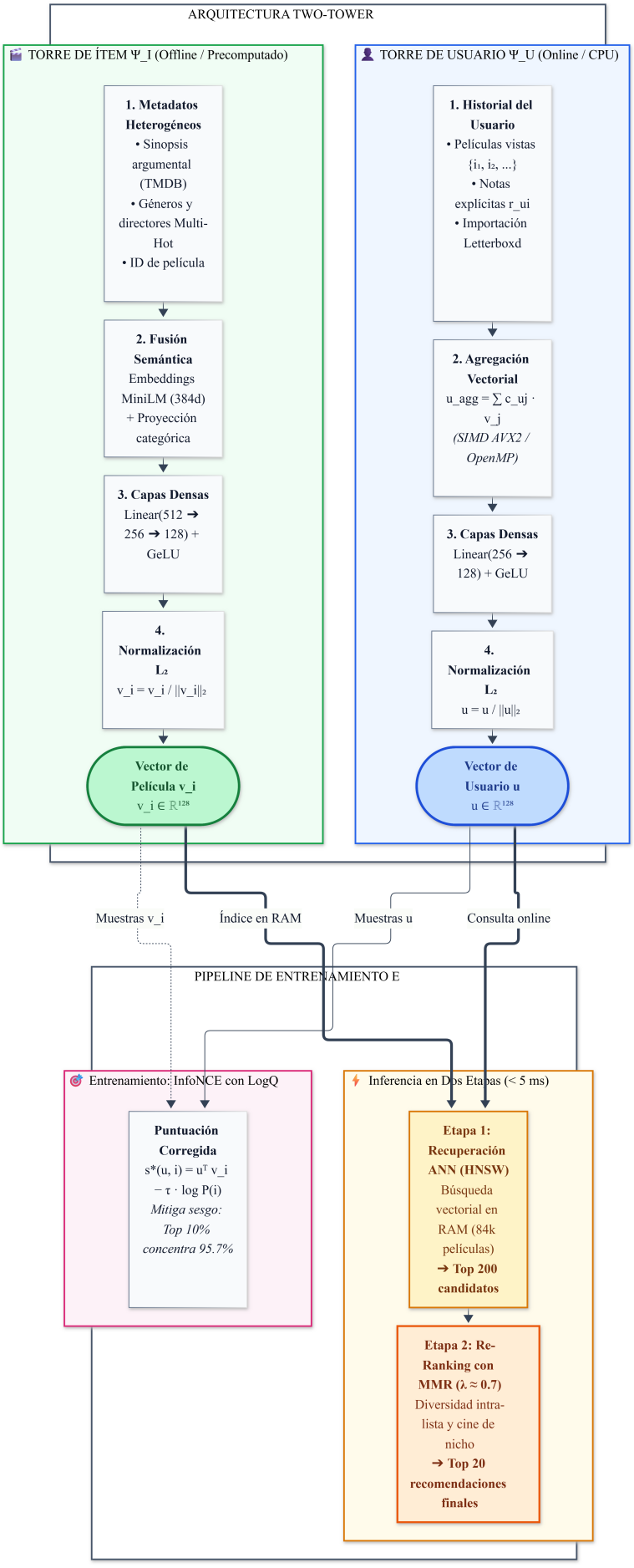


Figura 3: Arquitectura propuesta: desacoplamiento en dos etapas (Recuperación Two-Tower con búsqueda vectorial ANN y Reordenamiento con diversificación MMR).

6. Conclusiones y Próximos Pasos

Esta propuesta establece la viabilidad técnica del motor de recomendación tras caracterizar empíricamente los retos de MovieLens 32M (dispersión del 99.81 % y el 95.70 % de interacciones concentradas en solo el 10 % del catálogo). La integración con los metadatos de TMDB y el formato de Letterboxd proporciona el contexto necesario para superar el arranque en frío en un entorno real.

La arquitectura Two-Tower complementada con reordenamiento MMR ofrece el equilibrio óptimo para el proyecto: desacopla el cálculo pesado, acota el uso de memoria RAM a decenas de megabytes y traslada las operaciones críticas a rutinas de álgebra vectorial idóneas para su implementación paralela en C++ con OpenMP y SIMD. En la próxima entrega (E2) se abordará el pipeline de preprocesamiento, la generación de embeddings y el desarrollo de los benchmarks de rendimiento en CPU.

Referencias

[1] Harper, F. M., & Konstan, J. A. (2015). *The MovieLens Datasets: History and Context*. ACM Transactions on Interactive Intelligent Systems (TiiS), 5(4), 1–19.

[2] Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. IEEE Computer, 42(8), 30–37.

[3] Covington, P., Adams, J., & Sargin, E. (2016). *Deep Neural Networks for YouTube Recommendations*. Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16), 191–198.

[4] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). *Neural Collaborative Filtering*. Proceedings of the 26th International Conference on World Wide Web (WWW '17), 173–182.

[5] Liang, D., Krishnan, R. G., Hoffman, M. D., & Jebara, T. (2018). *Variational Autoencoders for Collaborative Filtering*. Proceedings of the 2018 World Wide Web Conference (WWW '18), 687–696.

[6] Kang, W. C., & McAuley, J. (2018). *Self-Attentive Sequential Recommendation*. IEEE International Conference on Data Mining (ICDM '18), 197–206.

[7] Steck, H. (2019). *Embarrassingly Shallow Autoencoders for Sparse Data*. Proceedings of the World Wide Web Conference (WWW '19), 3251–3257.

[8] Yi, X., Yang, J., Hong, L., Cheng, D. Z., Heldt, L., Kumthekar, A., Zhao, Z., Wei, L., & Chi, E. (2019). *Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations*. Proceedings of the 13th ACM Conference on Recommender Systems (RecSys '19), 269–277.

[9] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). *LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation*. Proceedings of the 43rd International ACM SIGIR Conference, 639–648.

[10] Carbonell, J., & Goldstein, J. (1998). *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*. Proceedings of the 21st Annual International ACM SIGIR Conference, 335–336.

[11] OpenMP Architecture Review Board. (2018). *OpenMP Application Programming Interface Version 5.0*. Technical Report.